

Milestone 4: Formal Methods-Inspired Static Analysis with Infer

CMPT 470/816: Advanced Software Engineering



Contents

1	Milestone Overview	2
2	Learning Objectives	2
3	Background	2
4	Tools Description	3
4.1	Infer	3
4.2	Semgrep	3
5	Required Datasets	3
5.1	Assigned Repositories	4
6	Experimental Tasks	5
6.1	Experiment 1: Tool Setup and Configuration (10 points)	5
6.2	Experiment 2: Infer Analysis and Metrics (30 points)	5
6.3	Experiment 3: Manual Triage of Findings (30 points)	6
6.4	Experiment 4: Stability Under Small Code Changes (15 points)	7
6.5	Experiment 5: Comparison with Semgrep (10 points)	8
7	Report Requirements	8
7.1	Introduction	8
7.2	Background	8
7.3	Experimental Setup	8
7.4	Results	8
7.5	Discussion	9
7.6	Lessons Learned	9
7.7	Conclusion	9
8	Deliverables	9
9	Grading Rubric	10
10	Important Notes	10
11	Tips for Success	10
12	Resources	11
12.1	Tool Documentation	11
12.2	Recommended Reading	11

1 Milestone Overview

Duration: Deadline is March 26, 2026

Weight: 11% of final grade

Software Engineering Area: Program behavior reasoning, path-sensitive analysis, formal-methods-inspired bug finding

This milestone focuses on reasoning about **possible program behaviors without executing the program**. Unlike rule-based analysis tools that flag violations of coding guidelines or style conventions, this milestone examines how path-sensitive, semantic analysis can identify concrete runtime failure scenarios such as null dereferences and resource leaks.

You will use Infer to analyze real software projects and study how the tool constructs execution-based explanations for potential failures. The emphasis is not on counting warnings, but on understanding **why a reported issue could occur**, what assumptions the analyzer makes, and how confident you are in the reported behavior.

This milestone introduces a different perspective on software analysis: instead of asking “Does this code violate a rule?”, you will ask “Is there a feasible execution in which this code fails?”

You will also compare Infer with a rule-based analyzer (Semgrep) in order to understand how **pattern-based analysis differs from behavioral reasoning**.

2 Learning Objectives

By completing this milestone, you will:

- Learn how automated tools reason about possible program executions without running the code
- Understand the difference between rule-based checking and path-sensitive semantic reasoning
- Gain experience interpreting execution traces and explanations produced by a formal-methods-inspired analyzer
- Analyze how null dereferences and resource leaks arise from interprocedural interactions
- Develop the ability to judge whether a reported issue represents a plausible runtime failure
- Evaluate the stability of behavioral analysis results under small, semantics-preserving code changes
- Compare behavioral analysis with rule-based pattern detection
- Reflect on the strengths and limitations of reasoning-based analysis in practical software engineering

3 Background

Static analysis examines code without executing it. Some analyzers use lightweight rule checks; others apply deeper reasoning inspired by formal methods.

Infer is a static analyzer that uses techniques such as:

- **Abstract Interpretation:** approximates program behavior to reason about possible executions
- **Separation Logic:** supports reasoning about heap-manipulating programs
- **Compositional Analysis:** analyzes functions in isolation and combines results

Infer does **not** prove programs correct. Instead, it identifies potential runtime failures such as null dereferences or resource leaks and provides a trace explaining how such failures could occur.

Rule-based analyzers such as Semgrep operate differently. They search for syntactic or semantic patterns that correspond to risky programming practices. These tools do not typically analyze feasible execution paths.

Understanding the strengths and limitations of these approaches is important for engineers deciding how to incorporate static analysis into development workflows.

4 Tools Description

4.1 Infer

URL: <https://github.com/facebook/infer>

Infer is an open-source static analyzer developed at Facebook (Meta). It detects issues such as null dereferences, resource leaks, and memory errors. Infer provides traces showing how a failure could arise along a specific execution path. Infer supports Java projects built with Maven or Gradle.

4.2 Semgrep

URL: <https://semgrep.dev>

Semgrep is a rule-based static analysis tool that identifies bug patterns and risky coding practices using syntactic matching rules. It is widely used for security analysis and maintainability checks.

5 Required Datasets

Each group has been assigned a small, self-contained Java repository. The repository is the **entire analysis scope**. You will run Infer on the full project and report on all findings. All assigned repositories use either a Maven wrapper or a Gradle wrapper.

If you encounter build or Infer errors that you cannot resolve, contact your TA with the full error log and your operating system and Java version. A replacement repository will be provided if needed.

5.1 Assigned Repositories

- **Group: Turn it to 11**
Assigned Repository: HikariCP
URL: <https://github.com/brettwooldridge/HikariCP>
- **Group: Tech Titans**
Assigned Repository: Gson
URL: <https://github.com/google/gson>
- **Group: Group 6**
Assigned Repository: jsoup
URL: <https://github.com/jhy/jsoup>
- **Group: TryingTXL**
Assigned Repository: Joda-Time
URL: <https://github.com/JodaOrg/joda-time>
- **Group: The Coders**
Assigned Repository: Apache Commons CSV
URL: <https://github.com/apache/commons-csv>
- **Group: `*(volatile team_t *)0 = "name"`**
Assigned Repository: Apache HttpComponents Client
URL: <https://github.com/apache/httpcomponents-client>
- **Group: Hard Coded**
Assigned Repository: Apache Commons Imaging
URL: <https://github.com/apache/commons-imaging>
- **Group: Vibe Engineers**
Assigned Repository: Jackson Databind
URL: <https://github.com/FasterXML/jackson-databind>
- **Group: Silicon Valley's Finest**
Assigned Repository: Feign
URL: <https://github.com/OpenFeign/feign>
- **Group: Future Business Majors**
Assigned Repository: Vavr
URL: <https://github.com/vavr-io/vavr>

- **Group: Hopefully Graduating**

Assigned Repository: Apache Commons Collections

URL: <https://github.com/apache/commons-collections>

- **Group: No Name** (Members: Peng Wang, Saima Moonsat, Alan Rony, Zhiyang Teng, Meghal)

Assigned Repository: Jackson Core

URL: <https://github.com/FasterXML/jackson-core>

- **Group: Team Power Rangers**

Assigned Repository: Apache Commons Lang

URL: <https://github.com/apache/commons-lang>

6 Experimental Tasks

Complete **all** experiments.

6.1 Experiment 1: Tool Setup and Configuration (10 points)

1. Install Infer (binary release, container image, or build from source).
2. Verify installation:

```
infer --version
```
3. Clone the assigned repository and verify it builds successfully.
4. Document the following:
 - Installation method and Infer version
 - Operating system
 - Java version
 - Build tool and version (Maven or Gradle, as applicable)
 - Repository URL and commit/tag used
 - Total lines of code in the repository
 - Setup challenges encountered and how they were resolved

6.2 Experiment 2: Infer Analysis and Metrics (30 points)

1. Run Infer using the command provided for your assigned repository.
2. Record the following metrics:
 - Total issues detected
 - Issue counts by category

- Analysis runtime (wall clock)
- Any build warnings or errors that affected the analysis

3. Fill in the summary table below.

Project	LOC	Total Issues	Top Issue Types	Runtime

Table 1: Infer Analysis Summary

6.3 Experiment 3: Manual Triage of Findings (30 points)

Select **two Infer issues** for detailed investigation:

- **Issue A:** one issue from the most frequent issue category
- **Issue B:** one issue from any different category

For each issue provide:

- Issue type and name as reported by Infer
- File and method name
- Relevant code snippet
- Screenshot or excerpt of the Infer trace

Execution Path Evidence Table (fill in once per issue):

Trace Step	File	Line	Explanation
Value introduced			
Conditional branch			
Risky state appears			
Failure location			

Table 2: Execution Path Evidence

Trace Understanding Questions

Answer the following for **each** of the two issues:

T1. First Feasible Failure Point

Infer reports the line where the failure occurs. Identify the **earliest program point** where the failure first becomes possible. Provide:

- File and line number
- Explanation of why this is the first risky state

T2. Infer Assumption

Identify one assumption Infer makes that allows the failure path to exist. Examples:

- a method may return null
- an exception may interrupt cleanup

- a branch condition may be feasible

Explain the assumption and cite the specific code location that supports it.

T3. Refactoring Robustness

Consider a small behavior-preserving refactor such as variable extraction, renaming, or reformatting. Would:

- Semgrep still detect the issue?
- Infer still report it?

Explain your reasoning based on the difference between pattern matching and path-sensitive analysis.

Classification

Classify each issue as one of:

- **Likely true defect**
- **Likely false positive**

Justify your classification using the surrounding code context.

Fix Validation

For **Issue A** only:

- Propose a concrete code-level fix
- Apply the fix to the source code
- Re-run Infer
- Report whether the issue disappears from the output

6.4 Experiment 4: Stability Under Small Code Changes (15 points)

Choose any one method in the repository. Apply one small semantics-preserving modification, such as:

- renaming a local variable
- adjusting whitespace or formatting
- extracting a sub-expression into a temporary variable

Re-run Infer and report:

- Did the total issue count change?
- Did the issues you investigated in Experiment 3 move or disappear?
- Did any new issues appear?
- What does this suggest about the stability of path-sensitive analysis?

6.5 Experiment 5: Comparison with Semgrep (10 points)

Run Semgrep on the same repository using the default Java ruleset

Collect:

- Total findings
- Top issue types
- Runtime

Tool	Total Findings	Top Issue Types	Runtime
Infer Semgrep			

Table 3: Tool Comparison

Discuss the following:

1. Did both tools report issues in the same files or methods?
2. Why might Semgrep detect issues that Infer does not?
3. Why might Infer detect issues that Semgrep does not?
4. Which tool appears more useful for detecting runtime failures in your assigned repository?
Justify your answer.

7 Report Requirements

Submit a **PDF report (5–7 pages)**.

7.1 Introduction

Overview of static analysis and behavioral bug detection.

7.2 Background

Concepts of approximation, false positives and negatives, and an overview of Infer and Semgrep.

7.3 Experimental Setup

Environment details, repository used, exact commands executed, and tool versions.

7.4 Results

Present results from all five experiments. Required figures:

- Bar chart of Infer issue counts by type
- Tool comparison table (Infer vs. Semgrep)

7.5 Discussion

Interpret your findings and compare the two analysis approaches.

7.6 Lessons Learned

Key insights about static analysis tools gained from this milestone.

7.7 Conclusion

Summary of findings and a recommendation on tool usage.

8 Deliverables

1. PDF Report

A complete written report following the structure in Section 7. All interpretations and conclusions must be supported by evidence in the submitted artifacts.

File name: `GroupName_Milestone4_report.pdf`

2. Raw Tool Outputs

All unfiltered outputs generated by each tool, including:

- `infer-out/report.json` from all Infer runs (baseline, fix validation, and stability experiment)
- Semgrep output in JSON format

Students must not delete, suppress, or post-process tool outputs except where explicitly documented in the report.

3. Execution Logs

Complete terminal logs for all tool executions, including exact commands, warnings, errors, and timing output.

4. Modified Source Files

The modified source file(s) used in Experiment 3 (fix validation) and Experiment 4 (stability check), clearly labeled.

5. Contribution and AI Usage Statement

A plain-text (`.txt`) file containing:

- Team name
- Milestone name
- Contribution of each group member (in percentages)
- AI tool usage by each member, or an explicit statement that no AI tools were used

6. Individual Contribution Report

Each group member must submit an individual contribution report *separately* to any TA by email. The report must include:

- Your name and NSID

- Team name
- Milestone name
- Your contribution (in percentages)
- A brief comment on your team members' contributions

9 Grading Rubric

Component	Points	Criteria
Experiment 1: Setup	10	Environment fully documented, build verified
Experiment 2: Analysis	30	Infer executed correctly, metrics recorded
Experiment 3: Triage	30	Evidence-based reasoning, correct trace interpretation, fix validated
Experiment 4: Stability	15	Clear before/after comparison with analysis
Experiment 5: Comparison	10	Insightful Infer vs. Semgrep comparison
Report Quality	5	Clear structure, required figures included
Contribution and AI Statement	5	Complete and honest
Individual Contribution Report	5	Submitted separately by each member
Total	110	

Table 4: Milestone 4 Grading Rubric

10 Important Notes

1. AI tools may only be used for writing assistance.
2. All experimental results must come from actual tool execution.
3. Fabricated or manually constructed data will result in a zero grade.
4. Use the same repository for all five experiments.
5. Record exact tool versions for reproducibility.

11 Tips for Success

- Start setup early — Infer installation can take time depending on your platform.
- Confirm the project builds cleanly before running Infer.
- Read the full Infer trace carefully when selecting issues for Experiment 3; a clear trace makes the triage questions much easier to answer.
- When running Semgrep, make sure to point it at the `src/main/java` directory, not the root, to avoid scanning test files.

12 Resources

12.1 Tool Documentation

- Infer: <https://fbinfer.com/docs/getting-started/>
- Semgrep: <https://semgrep.dev/docs/>

12.2 Recommended Reading

- Calcagno, C. et al. (2015). Moving Fast with Software Verification. In: Havelund, K., Holzmann, G., Joshi, R. (eds) NASA Formal Methods. NFM 2015. Lecture Notes in Computer Science, vol 9058. Springer, Cham. https://doi.org/10.1007/978-3-319-17524-9_1
- Bessey, A. et al. (2010). A few billion lines of code later: using static analysis to find bugs in the real world. *Communications of the ACM*, 53(2), 66–75. <https://doi.org/10.1145/1646353.1646374>
- Widyasari, R. et al. (2020). BugsInPy: a database of existing bugs in Python programs to enable controlled testing and debugging studies. In *Proceedings of ESEC/FSE 2020*, 1556–1560. <https://doi.org/10.1145/3368089.3417943>
- Johnson, B., Song, Y., Murphy-Hill, E., and Bowdidge, R. (2013). Why don't software developers use static analysis tools to find bugs? In *Proceedings of ICSE 2013*, 672–681. <https://doi.org/10.1109/ICSE.2013.6606613>
- Hanam, Q., Brito, F., and Mesbah, A. (2016). Discovering bug patterns in JavaScript. In *Proceedings of FSE 2016*, 144–156. <https://doi.org/10.1145/2950290.2950308>